

Pendulum

March 19, 2026

1 Assignment 1 - Stoffel - Chen Zekai

1.1 1. Environment Selection

For this assignment, I selected the **Pendulum-v1** environment from the Gymnasium library.

- **Environment Description:** The system consists of a pendulum attached at one end to a fixed point. The starting state is a random angle in $[-\pi, \pi]$ and a random angular velocity in $[-1, 1]$. The goal is to apply torque to swing it into an upright position. The episode truncates at 200 time steps.
- **Reason for Selection:** Pendulum-v1 is a classic problem in control theory. It has a continuous action space (torque) and a continuous observation space (x-y coordinates and angular velocity). Its reward function is defined as $r = -(\theta^2 + 0.1 \cdot \theta_{dt}^2 + 0.001 \cdot \text{torque}^2)$. This setup makes it a great choice for testing the SAC algorithm.

```
[ ]: import gymnasium as gym
from stable_baselines3 import SAC

# Initialize the Pendulum environment
env = gym.make("Pendulum-v1")

# Instantiate the SAC model
model = SAC("MlpPolicy", env, verbose=1)
```

Using cpu device

Wrapping the env with a `Monitor` wrapper

Wrapping the env in a DummyVecEnv.

1.2 2. Training the Agent

I used the SAC (Soft Actor-Critic) algorithm from the `stable_baselines3` library. SAC is an off-policy maximum entropy deep reinforcement learning algorithm. It is highly efficient for environments with continuous action spaces. The agent was trained for 20,000 timesteps, which is sufficient to learn the pendulum swing-up task.

```
[ ]: # Start training
print("Training started...")

# Train the agent for 20,000 timesteps
```

```

model.learn(total_timesteps=20000)

print("Training completed!")

# Save the trained model
model.save("sac_pendulum_model")
print("Model saved successfully!")

env.close()

```

```

-----
| rollout/          |          |
|   ep_len_mean    | 200      |
|   ep_rew_mean    | -383     |
| time/            |          |
|   episodes       | 76       |
|   fps            | 106      |
|   time_elapsed   | 142      |
|   total_timesteps | 15200    |
| train/           |          |
|   actor_loss     | 43.2     |
|   critic_loss    | 0.975    |
|   ent_coef       | 0.0694   |
|   ent_coef_loss  | -0.381   |
|   learning_rate  | 0.0003   |
|   n_updates      | 15099    |
-----

```

```

-----
| rollout/          |          |
|   ep_len_mean    | 200      |
|   ep_rew_mean    | -370     |
| time/            |          |
|   episodes       | 80       |
|   fps            | 107      |
|   time_elapsed   | 148      |
|   total_timesteps | 16000    |
| train/           |          |
|   actor_loss     | 44.2     |
|   critic_loss    | 0.963    |
|   ent_coef       | 0.0616   |
|   ent_coef_loss  | -0.404   |
|   learning_rate  | 0.0003   |
|   n_updates      | 15899    |
-----

```

```

-----
| rollout/          |          |
|   ep_len_mean    | 200      |
|   ep_rew_mean    | -358     |

```

time/		
episodes	84	
fps	107	
time_elapsed	155	
total_timesteps	16800	
train/		
actor_loss	42.7	
critic_loss	0.72	
ent_coef	0.0555	
ent_coef_loss	-0.462	
learning_rate	0.0003	
n_updates	16699	

rollout/		
ep_len_mean	200	
ep_rew_mean	-352	
time/		
episodes	88	
fps	108	
time_elapsed	162	
total_timesteps	17600	
train/		
actor_loss	46.6	
critic_loss	0.89	
ent_coef	0.0487	
ent_coef_loss	-0.209	
learning_rate	0.0003	
n_updates	17499	

rollout/		
ep_len_mean	200	
ep_rew_mean	-341	
time/		
episodes	92	
fps	109	
time_elapsed	168	
total_timesteps	18400	
train/		
actor_loss	36.1	
critic_loss	1.25	
ent_coef	0.0415	
ent_coef_loss	-0.389	
learning_rate	0.0003	
n_updates	18299	

rollout/		
ep_len_mean	200	
ep_rew_mean	-332	
time/		
episodes	96	
fps	109	
time_elapsed	175	
total_timesteps	19200	
train/		
actor_loss	39.1	
critic_loss	1.1	
ent_coef	0.0357	
ent_coef_loss	-0.173	
learning_rate	0.0003	
n_updates	19099	

rollout/		
ep_len_mean	200	
ep_rew_mean	-325	
time/		
episodes	100	
fps	109	
time_elapsed	182	
total_timesteps	20000	
train/		
actor_loss	36.9	
critic_loss	0.484	
ent_coef	0.0314	
ent_coef_loss	-1.22	
learning_rate	0.0003	
n_updates	19899	

Training completed!
Model saved successfully!

1.3 3. Testing and Rendering

I test the agent using `render_mode="human"`. The trained SAC model outputs the torque. This allows me to visually check if the pendulum swings up and keeps its balance.

```
[ ]: import gymnasium as gym
      from stable_baselines3 import SAC

      # Load the trained model and render the animation
      model = SAC.load("sac_pendulum_model")
      test_env = gym.make("Pendulum-v1", render_mode="human")
      obs, info = test_env.reset()
```

```
for i in range(5000):
    action, _states = model.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, info = test_env.step(action)

    if terminated or truncated:
        obs, info = test_env.reset()

test_env.close()
```

1.4 4. Experimental Result

Screenshot of the rendering test: